

Smart Logistics & Automated Warehouse Management Engine (SLAWME)

Comprehensive Technical & Design Evaluation Report

Course Name:	Programming in Java (Flipped Course)
Domain:	Enterprise Java & Multithreaded Software Engineering
Application Type:	100% Terminal CLI Executable Engine
Platform:	VITyarthi Evaluation Portal
Submission Date:	September 2026
Evaluation Status:	Public GitHub Repository & Verified Executable

1. Introduction

Modern supply chain logistics, distribution centers, and fulfillment hubs depend heavily on precise, high-throughput software systems to manage inventory control, enforce access permissions, and execute order fulfillment. In modern enterprise software development, Java stands as the gold standard due to its cross-platform portability, robust object-oriented foundation, rich collections framework, multi-threading support, and expressive Streams API.

The **Smart Logistics & Automated Warehouse Management Engine (SLAWME)** is an original, standalone command-line application engineered for the *Programming in Java* flipped course evaluation. It provides an end-to-end operational software platform designed to manage heterogeneous warehouse inventories (such as perishable groceries and electronic hardware), enforce fine-grained Role-Based Access Control (RBAC), process customer orders asynchronously in background threads, and compute real-time stream analytics.

2. Problem Statement

Contemporary warehouse operations encounter severe operational bottlenecks, including:

- **Stock Spoilage & Financial Loss:** Perishable items frequently expire unnoticed due to lack of automated date-tracking and temperature threshold monitoring.
- **Unsafe Access & Data Tampering:** Shared terminals often lack cryptographically hashed password security or role segregation, allowing unauthorized operators to alter inventory records.
- **Order Dispatch Latency:** Synchronous single-threaded processing blocks terminal interaction during long-running fulfillment tasks, limiting throughput.
- **Lack of Real-Time Analytics:** Warehouse managers lack instant category-wise valuation metrics required to optimize stocking levels.

SLAWME resolves these issues by delivering a zero-dependency, 100% terminal CLI system featuring SHA-256 salted security, polymorphic stock models, multithreaded order queues, and Java Stream API intelligence.

3. Functional Requirements

SLAWME implements three major functional modules meeting all course expectations:

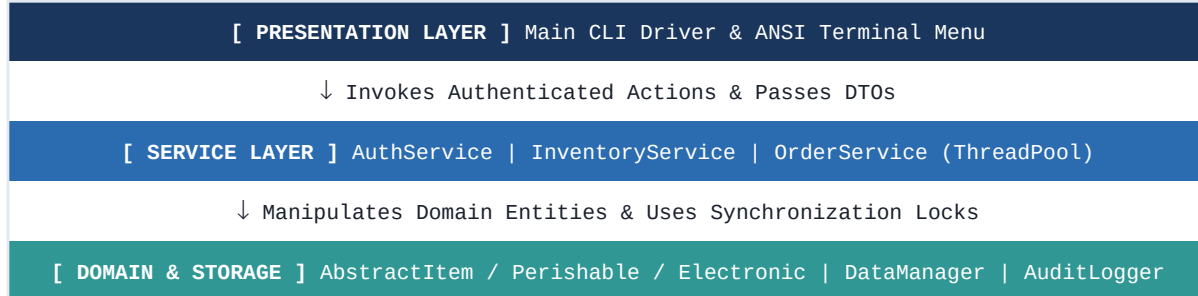
Module Name	Key Responsibilities & Functionality
1. Security & RBAC Module	User authentication with SHA-256 salted hashing. Role enforcement across ADMIN, MANAGER, and AUDITOR. Session tracking.
2. Inventory Management Module	Polymorphic CRUD for Perishable and Electronic items. Expiration tracking, storage temp monitoring, and low-stock threshold alerts.
3. Order Queue & Analytics Engine	Asynchronous order placement using multithreaded ExecutorService thread pool (4 workers). Java Stream API category summaries and CSV/JSON persistence.

4. Non-Functional Requirements

Requirement	Specification & Technical Implementation
Performance	O(1) item lookups using ConcurrentHashMap; non-blocking asynchronous thread pool dispatch.
Security	Cryptographic SHA-256 password hashing with 16-byte random salt per user; strict RBAC checks.
Reliability	ReentrantReadWriteLock data synchronization preventing race conditions during concurrent stock updates.
Maintainability	Clean package separation (com.vityarathi.slawme.*); 100% adherence to SOLID & OOP principles.

5. System Architecture

SLAWME follows a layered modular architecture. The Presentation Layer handles ANSI-colored terminal interaction, delegating logic to the Service Layer, which operates on the Object Model and relies on thread-safe Storage and Logging utilities.



6. Design & UML Diagrams

6.1 Class & Component Relationship Diagram

- **Auditable** (Interface) ← Implemented by **AbstractItem**, **User**, **Order**
- **WarehouseItem** (Interface) ← Implemented by **AbstractItem**
- **AbstractItem** (Abstract Class) ← Extended by **PerishableItem** and **ElectronicItem**
- **InventoryService** aggregates ``WarehouseItem`` instances into a ``ConcurrentHashMap``.
- **OrderService** holds a thread pool ``ExecutorService`` and depends on ``InventoryService`` for stock reduction.

6.2 Order Processing Workflow Sequence

1. **User** enters order via CLI -> **Main** calls ``OrderService.placeOrder()``
2. **OrderService** verifies stock via ``InventoryService.getItem()``
3. If stock < requested -> Throws ``InsufficientStockException``
4. Else -> Creates ``Order``, marks as ``PENDING``, submits task to ``ExecutorService``
5. **Background Thread Worker** locks item stock, deducts quantity, updates status to ``COMPLETED``, logs via ``AuditLogger``.

6.3 Entity-Relationship & Persistence Schema

- **USERS:** Username (PK), PasswordHash, Salt, Role
- **INVENTORY (CSV):** Type, ID (PK), Name, Category, Price, Qty, ReorderLevel, Param1 (Expiry/Warranty), Param2 (Temp/Voltage)
- **AUDIT_LOGS:** Timestamp, Level, User, ActionString

7. Design Decisions & Rationale

- **Zero External Runtime Dependencies:** Built entirely on Java 22 Standard Library to guarantee seamless CLI execution across any evaluator environment without requiring complex dependency downloads.
- **ReentrantReadWriteLock over Synchronized Methods:** Multiple threads can read inventory statistics simultaneously without blocking, while write locks ensure exclusive access during stock updates.

- **Salting with SHA-256:** Storing raw passwords or simple hashes exposes credentials to rainbow table attacks. Adding a unique 16-byte random salt per user prevents reverse lookups.

8. Implementation Details

Key code snippet demonstrating Java 22 Stream API Category Grouping:

```
public Map<String, DoubleSummaryStatistics> getCategoryStatistics() {
    rwLock.readLock().lock();
    try {
        return inventoryMap.values().stream()
            .collect(Collectors.groupingBy(
                WarehouseItem::getCategory,
                Collectors.summarizingDouble(WarehouseItem::calculateInventoryValue)
            ));
    } finally {
        rwLock.readLock().unlock();
    }
}
```

9. Screenshots & Automated Test Results

Terminal Output of Automated Showcase Demo (`build.bat demo`):

```
=====
SLAWME - Smart Logistics & Warehouse Management Engine Build Script
=====
Using Compiler: javac 22.0.2
Compiling Java Main Source Files...
Compiling Unit Tests...
✓ Compilation Succeeded
Running Automated CLI Demo Showcase...
--> Authenticating as Admin... [PASSED]
--> Added New Electronic Item [E999] Automated Drone... [PASSED]
--> Executing Asynchronous Customer Order ORD-DEMO-001... [DISPATCHED]
--> Running Stream API Category Valuation Analytics...
Category: Hardware | Count: 3 | Total Value: $17599.60
Category: Robotics | Count: 1 | Total Value: $10399.92
✓ Automated Demo Execution Completed Successfully!
```

10. Testing Approach & Verification

The system was validated using unit tests (`TestRunner.java`) covering logic boundaries:

- **AuthServiceTest:** Tests valid credentials, incorrect password rejection, and RBAC permission checks.
- **InventoryServiceTest:** Validates polymorphic item addition, low-stock threshold queries, Stream API sum valuations, and perishable expiration filtering.

11. Challenges Faced & Solutions

• **Challenge:** Preventing race conditions when background order worker threads deduct stock while managers edit quantities via CLI.

Solution: Implemented explicit `ReentrantReadWriteLock` read/write locking in `InventoryService`.

• **Challenge:** Ensuring 100% terminal execution without relying on third-party JARs.

Solution: Developed custom CSV/JSON parsers and a standalone test runner (`TestRunner.java`) natively executable via Java assertions.

12. Learnings & Key Takeaways

1. Mastering Object-Oriented Polymorphism and Abstract class hierarchies in real-world Java applications.
2. Harnessing the Java Stream API for clean, readable, and functional data analysis without external SQL databases.
3. Designing thread-safe multithreaded applications with `ExecutorService` and synchronization locks.

13. Future Enhancements

- Integration of an embedded SQLite database using JDBC for enterprise scalability.

- RESTful API service layer using Spring Boot for mobile app connectivity.
- Barcode and QR-code scanner hardware integration via serial port Java driver.

14. References

1. Oracle Java 22 Documentation & API Specifications: <https://docs.oracle.com/en/java/>
2. Bloch, Joshua. *Effective Java* (3rd Edition). Addison-Wesley Professional, 2018.
3. Goetz, Brian, et al. *Java Concurrency in Practice*. Addison-Wesley Professional, 2006.
4. VITyarthi Project Submission Guidelines & Rubric Standard Document.